12 INTERFACES IN TYPESCRIPT (AND WHERE THEY FIT NEXT TO DTOs)

An interface is a TypeScript-only description of a SHAPE — what properties/methods an object must have, and their types. It contains no implementation, no logic, and (this is the important part) it does not exist anymore once your code is compiled to JavaScript.

```
.ts file: interface Task { id: number; title: string }
                    |
            compiled by tsc
                    |
                    v
.js file: (nothing — the interface is erased completely)
```

REAL EXAMPLE FROM THIS PROJECT

```ts
// task.service.ts
export interface Task {
  id: number;
  title: string;
  description?: string;   // the '?' makes this property OPTIONAL
  done: boolean;
}

@Injectable()
export class TaskService {
  private tasks: Task[] = [
    { id: 1, title: 'Learn NestJS DTOs', done: false },
    { id: 2, title: 'Build task module', done: false },
  ];
  ...
}
```

- `Task` describes what every object in the `tasks` array must look like.
- If you tried `this.tasks.push({ id: 3, title: 5, done: false })` (title as a number), TypeScript refuses to compile — that's the whole value of the interface: catching shape mistakes BEFORE runtime.
- It's exported (`export interface Task`) so other files — like task.controller.ts — can use it too, e.g. to type a return value.

WHY INTERFACES EXIST — THE PROBLEM THEY SOLVE

Without an interface, you'd either:

1. Write the same object shape inline everywhere (`{ id: number; title: string; ... }`) and repeat yourself, or
2. Use `any` , which throws away type-checking entirely — typos like `task.titel` would silently return `undefined` instead of failing at compile time.

An interface names the shape once, so:

- You write it once and reuse it (`Task[]` , `Task | null` , function params typed as `Task` , etc.)
- Autocomplete/IntelliSense knows exactly which properties exist.
- Renaming/adding a field is a single edit that TypeScript then verifies everywhere the interface is used.

HOW TO DECLARE ONE

```
interface Point {
  x: number;
  y: number;
}

// optional property
interface Point3D {
  x: number;
  y: number;
  z?: number;        // may be omitted
}

// readonly property (can't be reassigned after creation)
interface Config {
  readonly apiKey: string;
}

// method signature
interface Greeter {
  greet(name: string): string;
}

// extending another interface
interface AdminUser extends User {
  permissions: string[];
}
```

A class can also formally promise to match an interface with `implements` :

```
interface Shape {
  area(): number;
}

class Circle implements Shape {
  constructor(private radius: number) {}
  area() {
    return Math.PI * this.radius ** 2;
  }
}
// TypeScript errors if Circle doesn't provide an area() method.
```

INTERFACE vs TYPE ALIAS (QUICK NOTE)

`type` can describe the same shapes and is often interchangeable:

```
type Task = { id: number; title: string; done: boolean };
```

Differences that matter in practice:

```
  - `interface` can be "reopened" and merged (declaration merging) —
    rarely used deliberately, but a quirk to know about.
  - `type` can describe things an interface can't: unions
    (`type Id = number | string`), tuples, mapped types.
  - Both are fully erased at compile time either way.
Rule of thumb: prefer `interface` for plain object/class shapes (like
`Task` above), reach for `type` when you need unions or other type-level
composition.
```

INTERFACE vs DTO (THIS IS THE PART THAT MATTERS HERE)

This project already shows both side by side:

```
task.service.ts        interface Task { id, title, description?, done }
dto/create-task.dto.ts class CreateTaskDto { title, description? }
```

They look similar but serve different jobs and CANNOT be swapped:

```
interface Task          — describes INTERNAL data already trusted to
                          be correct (what's sitting in the `tasks`
                          array in memory). Pure compile-time type,
                          zero runtime footprint, zero validation.

class CreateTaskDto     — describes data arriving from OUTSIDE the
                          app (the request body) that has NOT been
                          trusted yet. Written as a real class (not
                          an interface) specifically so
                          @IsString()/@IsNotEmpty() decorators from
                          class-validator have something to attach
                          to at runtime, and ValidationPipe can
                          instantiate + check it before your handler
                          runs. See theory/13-dto.txt.
```

An interface CANNOT be decorated with @IsString() etc. — decorators attach to real classes, and interfaces don't exist at runtime, so there's nothing there for a decorator to attach to. That's precisely why DTOs are written as classes even though "just an interface" would look simpler.

★ RULE OF THUMB

Reach for an interface to describe the shape of data you already trust (service-internal models, function parameters, return types). Reach for a DTO class instead the moment that shape needs to cross a boundary (HTTP request body) where the data must be validated, not just typed.

See also: theory/13-dto.txt, theory/09-pipes.txt